INT1405 - DISTRIBUTED SYSTEMS

LECTURE 06
Consistency and Replication

Semester 1, 2026-2027

Lê Hà Thanh
lehathanh@ptithcm.edu.vn



Tanenbaum & van Steen (2nd Edition),
Chapter 7 (Sections 7.1–7.5)

Slide 02: Learning Objectives & Operational Roadmap

Pedagogical Goals

- Distinguish Data-Centric consistency models (Strict, Sequential, Causal, FIFO, Weak, Release, Entry).
- Evaluate Client-Centric session guarantees (Monotonic Reads/Writes, Read-Your-Writes, Writes-Follow-Reads).
- Solve Gifford's Weighted Quorum equations ($N_R + N_W > N \; and \; N_W > N/2$) for consistency bounds.
- Compare Replica Management & Content Propagation algorithms (Push vs. Pull, Epidemic Gossiping).

Module Progression Roadmap

- Module 1: Introduction & Data-Centric Consistency Models (Slides 3–16).
- Module 2: Client-Centric Consistency Models & Session Guarantees (Slides 17–26).
- Module 3: Replica Management, Content Placement & Gossip Protocols (Slides 27–34).
- Module 4: Consistency Protocols, Primary-Backup & Quorum Systems (Slides 35–38).
- Module 5: Real-World Case Studies, Review & Practical Lab Specifications (Slides 39–40).

Content

- Module 1: Introduction & Data-Centric Consistency Models (Slides 3–16).
- Module 2: Client-Centric Consistency Models & Session Guarantees (Slides 17–26).
- Module 3: Replica Management, Content Placement & Gossip Protocols (Slides 27–34).
- Module 4: Consistency Protocols, Primary-Backup & Quorum Systems (Slides 35–38).
- Module 5: Real-World Case Studies, Review & Practical Lab Specifications (Slides 39–40).

Slide 03: Reasons for Replication: Fault Tolerance & Performance

Reliability Enhancement

- **Data High Availability**: Replicating data across independent server nodes prevents data loss when physical disks or hardware fail.
- **Fault Masking**: If 1 server crashes, the system seamlessly redirects read and write requests to surviving active replicas.
- **Geographic Redundancy**: Protects business operations against catastrophic regional power outages or datacenter disasters.

Performance & Scaling Bottlenecks

- **Latency Minimization**: Positioning replicas near end users drastically reduces wide-area network propagation delays.
- **Read Scalability**: Distributes high query volumes across N server nodes to prevent CPU and network interface card saturation.
- **Write Trade-Off**: Scaling reads via replication introduces update propagation overhead across all replicas.

Slide 04: The Replication Scalability Bottleneck & Consistency Trade-offs

The Consistency Dilemma

- **Synchronous Overhead**: Keeping replicas perfectly identical requires atomic write broadcasts and global locking across all nodes.
- **Latency Degradation**: Synchronous wide-area network messages cause write performance to degrade as node count N scales.
- **CAP Theorem Trade-Off**: A distributed system cannot simultaneously provide Consistency, Availability, and Partition Tolerance.

Relaxing Consistency Standards

- **PACELC Theorem Extension**: If Partition (P) occurs, trade off Availability (A) or Consistency (C); Else (E), trade off Latency (L) or Consistency (C).
- **Weak Consistency Advantage**: Relaxing strict consistency allows local writes to return instantly, converging replicas asynchronously.
- **Domain-Driven Selection**: Financial ledgers require strict consistency, whereas social media feeds prefer high availability.

Slide 05: Data-Centric Consistency Models: Contract Abstraction

Contract Abstraction Concept

- **Formal Agreement**: A data-centric consistency model acts as a contract between a distributed data store and executing process threads.
- **Predictable Semantics**: If applications follow specific rules when issuing reads/writes, the data store guarantees deterministic results.
- **System View**: Focuses on the global state of data items across all server storage replicas simultaneously.

Standard Execution Notation

- $W_i(x)a$: Process P_i performs a Write operation updating data item x to value 'a'.
- $R_i(x)b$: Process P_i performs a Read operation on data item x, returning value 'b'.
- **Space-Time Ordering**: Operations advance from left to right along horizontal process execution timelines.

Slide 06: Strict Consistency: Absolute Physical Time Ordering

Formal Definition

- **Physical Time Guarantee**: Any read operation on data item x returns a value corresponding to the result of the most recent write on x in physical time.
- **Mathematical Condition**: Read $R_i(x)$ must return value 'a' of the latest write $W_j(x)a$ across absolute physical time t.
- **Zero Delay Ideal**: Modifications take effect instantaneously across all global replicas everywhere on Earth.

Practical Impossibility

- **Physical Limits**: Demands perfectly synchronized hardware clocks and faster-than-light message delivery across networks.
- **Asynchronous Reality**: Network transmission latencies prevent real-world systems from guaranteeing strict physical consistency.
- **Theoretical Baseline**: Serves as a reference baseline against which weaker real-world models are evaluated.

Slide 07: Sequential Consistency (Lamport, 1979)

Formal Definition

- **Interleaved Sequence**: The result of any execution is the same as if operations of all processes were executed in some sequential order.
- **Program Order Invariant**: Operations of each individual process appear in this sequence in the order specified by its program.
- **Global Agreement**: Every replica node in the system must observe the exact same interleaved sequence of operations.

Key Properties & Decoupling

- **Physical Time Independence**: Operations do not need to be ordered by physical time, only agreed upon globally across all nodes.
- **Interleaving Freedom**: Multiple valid interleavings exist as long as process local program order is strictly preserved.
- **Implementation Cost**: Requires global coordination (e.g., totally-ordered multicast) to enforce global sequence agreement.

Slide 08: Evaluating Sequential Consistency Execution Traces

Trace Example A (Valid Sequential)

- P_1: $W(x)a$ | P_2: $W(x)b$
- P_3: $R(x)b \rightarrow R(x)a$ | P_4: $R(x)b \rightarrow R(x)a$

- **Analysis**: Valid! Both P_3 and P_4 agree that write 'b' occurred before write 'a' in global sequence $W(x)b \rightarrow W(x)a$.

Trace Example B (Invalid Sequential)

- P_3: $R(x)b \rightarrow R(x)a$
- P_4: $R(x)a \rightarrow R(x)b$

- **Analysis**: Invalid! P_3 observes write 'b' first, whereas P_4 observes write 'a' first. Disagreement violates global interleaving.

Slide 09: Causal Consistency: Preserving Cause and Effect

Formal Definition

- **Causal Order**: Writes that are causally related must be seen by every process in the system in the exact same order.
- **Concurrent Order**: Concurrent writes that are not causally related may be observed in different orders by different processes.
- **Weaker Guarantee**: Relaxes global ordering for independent operations, drastically improving throughput and responsiveness.

Establishing Causal Dependency

- **Read-After-Write Dependency**: If P_2 reads $R(x)a$ written by P_1 and subsequently issues $W(x)b$, then $W(x)a$ causally precedes $W(x)b$.
- **Transitive Dependency**: If $W(x)a \rightarrow W(x)b$ and $W(x)b \rightarrow W(x)c$, then $W(x)a$ causally precedes $W(x)c$.
- **Causal Graphs**: Vector timestamps or Lamport clocks track explicit causal relationships between events.

Slide 10: Trace Analysis: Causally Related vs. Concurrent Writes

Trace 1 (Causally Linked Execution)

- P_1: $W(x)a$
- P_2: $R(x)a \rightarrow W(x)b$ (Causally dependent: $W(x)a \rightarrow W(x)b$)
- P_3: $R(x)a \rightarrow R(x)b$ | P_4: $R(x)a \rightarrow R(x)b$

- **Rule**: Both P_3 and P_4 must observe 'a' before 'b' because P_2 read 'a' before issuing 'b'.

Trace 2 (Concurrent Write Execution)

- P_1: $W(x)a$ | P_2: $W(x)b$ (Concurrent: P_2 did NOT read 'a')
- P_3: $R(x)a \rightarrow R(x)b$ | P_4: $R(x)b \rightarrow R(x)a$

- **Rule**: Valid under Causal Consistency! Since $W(x)a \,||\, W(x)b$, P_3 and P_4 can observe them in opposite relative orders.

Slide 11: FIFO (Pipelined RAM / PRAM) Consistency

Formal Definition

- **Process-Order FIFO**: Writes issued by a single process are seen by all other processes in the order in which they were issued.
- **Inter-Process Independence**: Writes issued by different processes may be seen in different relative orders by different processes.
- **Pipelined Delivery**: Corresponds to streaming updates through independent FIFO message queues between processes.

Implementation Advantages

- **Zero Cross-Process Synchronization**: Process updates do not block waiting for peer process coordination.
- **High Performance**: Extremely fast write throughput over wide-area network socket connections.
- **Simple Queue Model**: Implemented natively using TCP channel sequence counters.

Slide 12: FIFO Consistency Trace Analysis & Banking Anomaly

Trace Evaluation (Valid FIFO)

- P_1: $W(x)a$
- P_2: $R(x)a \rightarrow W(x)b$
- P_3: $R(x)b \rightarrow R(x)a$ | P_4: $R(x)a \rightarrow R(x)b$

- **Analysis**: Valid under FIFO! Writes from P_1 and P_2 arrive at P_3 and P_4 in different orders.

The Banking Anomaly under FIFO

- *Initial Balance*: \$100. P_1 deposits \$20 (W(bal)120). P_2 adds 10% interest (W(bal)110).
- *Replica Divergence*: Replica 1 executes Deposit then Interest (\$120 -> \$132). Replica 2 executes Interest then Deposit (\$110 -> \$130).
- ***Conclusion***: Weak FIFO consistency allows financial balance divergence, making it unsuitable for bank ledgers.

Slide 13: Weak Consistency: Grouping Operations with Synchronization

Concept & Motivation

- **Coarse-Grained Overhead Reduction**: Enforcing global consistency on every individual read/write is inefficient.
- **Synchronization Variables**: Introduces explicit barrier primitives (S) to flush and synchronize memory in batches.
- **Batch Execution**: Allows fast local memory reads and writes between synchronization points.

3 Core Rules of Weak Consistency

- **Rule 1**: Accesses to synchronization variables (S) are sequentially consistent.
- **Rule 2**: No access to a synchronization variable is permitted until all previous writes have completed everywhere.
- **Rule 3**: No access to data (read/write) is permitted until all previous accesses to synchronization variables have completed.

Slide 14: Weak Consistency Trace Analysis & Memory Barriers

Valid Synchronization Sequence

- P_1: $W(x)a \rightarrow W(x)b \rightarrow S$
- P_2: $S \rightarrow R(x)b$

- **Explanation**: Valid! Synchronization variable S acts as a memory fence, ensuring P_1's writes complete before P_2 reads.

Invalid Synchronization Sequence

- P_1: $W(x)a \rightarrow W(x)b \rightarrow S$
- P_2: $R(x)a \rightarrow S$

- **Explanation**: Invalid! P_2 attempts to read data item x before executing synchronization variable S, returning stale data.

Slide 15: Release Consistency: Explicit Acquire/Release Semantics

Acquire & Release Primitives

- `Acquire(L)`: Requests lock L. Ensures local process memory is updated with all global writes associated with lock L.
- `Release(L)`: Releases lock L. Forces all local updates performed within the critical section to propagate to all replicas.
- **Scoped Synchronization**: Only memory modified inside the critical section guarded by lock L is synchronized.

Eager vs. Lazy Release Mechanics

- **Eager Release Consistency**: Pushes dirty memory updates to all network replicas immediately when `Release(L)` is called.
- **Lazy Release Consistency**: Delays pushing updates until another process executes an `Acquire(L)` on the exact same lock.
- **Bandwidth Savings**: Lazy release eliminates unnecessary network broadcasts if no other process requests the lock.

Slide 16: Entry Consistency: Binding Locks to Specific Data Items

Concept & Granularity

- **Object-Lock Association**: Associate every individual data item or object explicitly with a specific synchronization lock L.
- **Targeted Synchronization**: Acquiring lock L1 synchronizes ONLY item x guarded by L1, ignoring unrelated database items.
- **Parallel Execution**: Process 1 locks item x while Process 2 locks item y, executing updates concurrently with zero interference.

Access Modes & Requirements

- **Exclusive Mode**: Required for write operations; no other process can hold the lock simultaneously.
- **Shared Mode**: Permitted for concurrent read-only operations across multiple reader processes.
- **Comparison**: Entry consistency provides finer granularity than Release consistency by avoiding full memory flushes.

Content

- Module 1: Introduction & Data-Centric Consistency Models (Slides 3–16).
- Module 2: Client-Centric Consistency Models & Session Guarantees (Slides 17–26).
- Module 3: Replica Management, Content Placement & Gossip Protocols (Slides 27–34).
- Module 4: Consistency Protocols, Primary-Backup & Quorum Systems (Slides 35–38).
- Module 5: Real-World Case Studies, Review & Practical Lab Specifications (Slides 39–40).

Slide 17: Client-Centric Consistency: Eventual Consistency

Eventual Consistency Foundations

- **Read-Heavy Environments**: Designed for systems with rare writes and massive read concurrency (e.g., DNS, Web Caches).
- **Convergence Guarantee**: If no new updates occur, eventually all replicas converge and return identical values when queried.
- **Temporary Divergence**: Accepts short-term replica divergence in exchange for ultra-low latency and 100% availability.

Mobile & Portable Client Model

- **Mobile Navigation**: A mobile user connects to different replica servers as they move physically across network locations.
- **Single-Client View**: Guarantees consistent data views for a single client process accessing different replicas over time.
- **Decoupled Peers**: Replicas do not need to be in sync with each other, as long as client session state is preserved.

Slide 18: Client-Centric Notation & Session Mechanics

Formal Session Notation

- $x_i(t)$: State of data item x on replica server S_i at physical time t.
- $WS(x_i[t])$: The Write Set representing the sequence of write operations performed on replica S_i up to time t.
- $WS(x_i[t1])$; $WS(x_j[t2])$: Write operations from replica S_i are propagated and applied to replica S_j.

Client Session Context

- **Session Abstraction**: Client middleware maintains session context vectors as clients switch server connections.
- **Write-Set Tracking**: Tracks the set of updates issued or observed by the client to enforce session guarantees.
- **Seamless Failover**: Allows mobile clients to reconnect to new replicas without experiencing data regression.

Slide 19: Monotonic Reads Guarantee

Formal Definition

- **No Time Reversal**: If a process reads value v1 of data item x, any successive read on x by that process will return v1 or a newer version.
- **Monotonic Time Constraint**: A client will never observe an older version of data than it has already seen in a previous read.
- **Replica Switching**: Enforced when a client reads from Server A and subsequently reads from Server B.

Real-World Violation Example

- **Email Inbox Scenario**: User reads inbox on Server A (sees 5 new emails). User refreshes page; request routes to slow Server B.
- **Violation Effect**: Server B returns 2 emails! User observes emails disappearing because Server B is lagging behind Server A.
- **Fix Requirement**: Server B must fetch Write Set $WS(A)$ before serving the client's read request.

Slide 20: Trace Analysis: Valid vs. Invalid Monotonic Reads

Valid Monotonic Read Sequence

• Client: $R(x_1) \rightarrow v_1$ [moves to Server S_2] $\rightarrow R(x_2) \rightarrow v_1 \text{ or } v_2$

• **Requirement**: Server S_2 applies Write Set $WS(x_1)$ before returning the read result to the client.

• **Outcome**: Client observes data staying constant or moving forward in time.

Invalid Monotonic Read Sequence

• Client: $R(x_1) \rightarrow v_2$ $[moves\ to\ Server\ S_2] \rightarrow R(x_2) \rightarrow v_1$ $(where\ v_1\ is\ older\ than\ v_2)$

• Violation: Server S_2 served a stale read missing the client's previously observed Write Set.

• Enforcement: Client library rejects stale responses or stalls until Server S_2 updates its write set.

Slide 21: Monotonic Writes Guarantee

Formal Definition

- **Serialized Client Writes**: A write operation by a process on data item x is completed on any replica before any successive write on x by the same process.
- **Order Preservation**: Successive writes issued by a single client are applied everywhere in the exact order they were issued.
- **Multi-Replica Serialization**: Guarantees write ordering even if Write 1 lands on Server A and Write 2 lands on Server B.

Real-World Violation Example

- **Software Deployment Scenario**: Developer uploads compiled library file (Write 1 on Server A), then updates config file (Write 2 on Server B).
- **Violation Effect**: Server C receives Write 2 before Write 1, causing application startup failure due to missing libraries.
- **Fix Requirement**: Server C must apply Write 1 before executing Write 2.

Slide 22: Monotonic Writes Protocol Mechanics

Protocol Enforcement

- **Header Dependency**: Client attaches a write-set dependency header (timestamp of Write 1) when issuing Write 2 on Server S_2.
- **Server Stalling**: Server S_2 delays executing Write 2 until it fetches and applies Write 1 from Server S_1 over the network.
- **State Alignment**: Ensures server state dominates client history before accepting new mutations.

Scope Comparison: Monotonic Writes vs Causal Consistency

- **Monotonic Writes**: Client-centric model enforcing write order for a SINGLE client process.
- **Causal Consistency**: Data-centric model enforcing write order across ALL causally dependent processes globally.
- **Overhead Difference**: Monotonic writes requires zero inter-client tracking.

Slide 23: Read-Your-Writes Guarantee & Sticky Routing

Formal Definition & UX Importance

- **Instant Update Feedback**: The effect of a write operation by a process on data item x will always be seen by a successive read on x by the same process.
- **User Expectation**: Users expect their own updates (e.g., password changes, profile edits) to take effect immediately.
- **Violation Example**: User updates password on Server A, reloads page on Server B, and gets 'Incorrect Password' error.

Enforcement via Sticky Sessions

- **Sticky Load Balancing**: Load balancers route all requests from a client session to the exact same server (Server A).
- **Write-Set Fetching**: If failover occurs, Server B fetches the client's write set from Server A before serving reads.
- **Trade-off**: Sticky sessions are simple but reduce load-balancing flexibility during server spikes.

Slide 24: Writes-Follow-Reads (Causal Client) Guarantee

Formal Definition

- **Causal Reaction**: A successive write operation by a process on data item x following a previous read on x by the same process occurs on the same or a newer version of x.
- **Causal Chain Preservation**: If a client reads value v1 and subsequently issues Write 2 based on v1, Write 2 is only applied to replicas that have processed v1.
- **Client Causality**: Serves as the client-centric equivalent of system-wide causal tracking.

Real-World Discussion Board Example

- **Forum Workflow**: User reads Post 1 (Write 1) on Server A, then posts Reply (Write 2) on Server B.
- **Violation Effect**: Server C receives Reply (Write 2) before Post 1 (Write 1), creating an orphan reply pointing to a non-existent parent thread.
- **Fix Requirement**: Server C holds Write 2 in buffer until Write 1 arrives.

Slide 25: Client-Centric Consistency Models Comparison

Consistency Model	Core Guarantee to Single Client Context	Violation Scenario Prevented
Monotonic Reads	Read values never move backward in logical or version time.	Page refresh shows older data than previously read.
Monotonic Writes	Successive writes issued by client executed in order.	App configuration update applied before library binary update.
Read-Your-Writes	Client always observes the effects of its own updates.	User updates password but page reload prompts old password.
Writes-Follow-Reads	Writes applied after reads execute on updated target state.	Discussion forum reply appears before the parent post is created.

Slide 26: Implementing Client-Centric Protocols & PACELC Trade-offs

Session Vector Timestamps

- **Dual Client Vectors**: Client tracks V_{read} (latest observed read version) and V_{write} (latest issued write version).
- **Server Validation**: Replica S_i accepts client request only if $V_{server} \geq V_{read}$ (or V_{write}). Otherwise, S_i stalls and fetches missing writes.
- **State Convergence**: Guarantees mathematically that client session constraints are preserved across arbitrary server hops.

PACELC Theorem & Consistency Spectrum

- **PACELC Formulation**: If Partition (P) → choose Availability (A) or Consistency (C); Else (E) → choose Latency (L) or Consistency (C).
- **System Classification**: Spanner = PC/EC (Strict); Cassandra = PA/EL (Tunable); MongoDB = PC/EC.
- **Engineering Rule**: Select the weakest consistency model that eliminates application-level user anomalies.

Content

- Module 1: Introduction & Data-Centric Consistency Models (Slides 3–16).
- Module 2: Client-Centric Consistency Models & Session Guarantees (Slides 17–26).
- Module 3: Replica Management, Content Placement & Gossip Protocols (Slides 27–34).
- Module 4: Consistency Protocols, Primary-Backup & Quorum Systems (Slides 35–38).
- Module 5: Real-World Case Studies, Review & Practical Lab Specifications (Slides 39–40).

Slide 27: Replica Management: Server Placement Frameworks

The Replica Management Problem

- **Server Placement**: Deciding where to physically deploy replica server nodes across network topology graphs.
- **Content Placement**: Deciding which specific data objects should be placed on which replica servers.
- **Optimization Goal**: Minimize client access latency while keeping network bandwidth and hardware costs low.

Greedy Server Placement Algorithm

- **Optimization Model**: Select K server locations out of N potential site locations to minimize average client distance.
- **Algorithmic Steps**: Compute latency matrix → Select 1st location minimizing total distance → Iteratively add sites maximizing distance reduction.
- **Infrastructure Nature**: Server placement is a long-term, static infrastructure decision.

Slide 28: Content Placement: Permanent, Server-Initiated & Client-Initiated

1. Permanent Replicas

- **Static Core Clusters**: Core database mirrors configured manually by system administrators (e.g., primary-secondary SQL clusters).
- **High Availability Base**: Forms the permanent backbone of enterprise database storage.

2. Server-Initiated Replicas (Push Caching)

- **Dynamic Edge Replicas**: Created automatically by servers to handle localized regional traffic spikes.
- **Access Threshold Monitoring**: If request volume from Region R exceeds threshold, server instantiates new edge replica near Region R.

3. Client-Initiated Replicas (Client Caches)

- **Browser & Proxy Caches**: Local caches managed entirely by client browsers or local HTTP proxies to eliminate redundant requests.

Slide 29: Content Propagation Paradigms: What to Transmit

1. Propagate Notification (Invalidation)

- **Invalidation Signals**: Send a short notification message informing replicas that data item x has been updated.
- **Cache Discard**: Replicas invalidate local cache entries. Subsequent reads trigger a fetch for fresh data.
- **Best For**: Low read-to-write ratios (frequent writes, rare reads). Saves network bandwidth.

2. Propagate Data / Full State

- **Full Object Transfer**: Transmit the entire updated data object payload to all replicas.
- **Best For**: High read-to-write ratios (rare writes, frequent reads). Avoids cache miss latency.

3. Propagate Operations (Active Replication)

- **Operation Delta**: Transmit update code/parameters (e.g., `Add(x, 5)`) rather than full state.
- **Best For**: Large data structures modified by small computational deltas.

Slide 30: Push-Based vs. Pull-Based Propagation Protocols

Push-Based Protocols (Server-Driven)

- **Proactive Broadcasting**: Master servers push updates to replicas immediately when write operations occur.
- **High Consistency Maintenance**: Keeps replicas closely synchronized with low read latency.
- **Overhead**: Incurs bandwidth costs even if updated data is never read by clients.
- **Primary Use Case**: Primary-backup database replication and critical state synchronization.

Pull-Based Protocols (Client-Driven)

- **Reactive Fetching**: Replicas request fresh updates from master servers only when client reads occur or local caches expire.
- **Low Bandwidth Cost**: Zero network overhead when data is unchanged.
- **Latency Impact**: Introduces round-trip delays on cache misses.
- **Primary Use Case**: Web browser caches and CDN edge proxy caching.

Slide 31: Epidemic Update Protocols: Gossip Architecture

Biological Spread Analogy

- **P2P Convergence**: Inspired by disease spread models. Designed for massive scale peer-to-peer networks with no central coordinator.
- **Random Exchange**: Nodes periodically select random peers to exchange data updates.
- **Scalability & Resilience**: Scales seamlessly to thousands of nodes with zero single-point-of-failure risk.

Node State Taxonomy

- **Infective Node**: Holds a fresh update and actively attempts to spread it to peer nodes.
- **Susceptible Node**: Has not yet received the update and is eligible to be updated.
- **Removed Node**: Holds the update but has stopped attempting to spread it to peers.

Slide 32: Epidemic Mechanics: Anti-Entropy vs. Rumor Spreading

Anti-Entropy Protocol

- **Periodic Sync**: Node P randomly selects peer Q and exchanges full write sets to synchronize state.
- **Push Mode**: P sends updates to Q | Pull Mode: Q fetches updates from P.
- **Push-Pull Mode**: P and Q exchange missing updates bidirectionally (fastest $O(\log N)$ rounds convergence).

Rumor Spreading (Gossiping)

- **Active Push**: When node P receives fresh update, it contacts random peer Q to push the rumor.
- **Termination Rule**: If Q was already infected, P stops spreading the rumor with probability $1/k$ (becomes Removed).
- **Traffic Control**: Prevents epidemic messages from circulating endlessly over the network.

Slide 33: Managing Deletions in Epidemic Systems: Death Certificates

The Erasure Paradox

- **Recreation Hazard**: If an item is deleted locally from replica P by erasing its record, a subsequent anti-entropy exchange with peer Q will treat the missing item as an un-received update and re-create it!
- **Distributed Conflict**: Deletions cannot be represented simply by absence of data.

Tombstone Deletion Protocol

- **Death Certificates**: Deletions write a special Tombstone marker tagging the item as deleted with a timestamp.
- **Epidemic Propagation**: Tombstones spread across peers via standard gossip exchanges, forcing all replicas to process deletion.
- **Garbage Collection**: Tombstones are retained for time $T_{cleanup}$, then permanently purged from disk storage.

Slide 34: Case Study: Content Delivery Network (CDN) Architecture

Global Caching Infrastructure

- **Edge Clusters**: Deploying proxy server clusters across worldwide Internet Service Providers (Cloudflare, Akamai).
- **Geographic Latency Hiding**: Delivers web assets to end users from edge caches positioned milliseconds away.

Anycast Routing & Cache Management

- **BGP Anycast Routing**: Routes client HTTP requests automatically to the physically closest operational edge server.
- **Static Assets**: Served via Pull caching with high Time-To-Live (TTL) expiration headers.
- **Dynamic Invalidation**: Origin servers push API invalidation calls to purge stale edge items instantly.

Content

- Module 1: Introduction & Data-Centric Consistency Models (Slides 3–16).
- Module 2: Client-Centric Consistency Models & Session Guarantees (Slides 17–26).
- Module 3: Replica Management, Content Placement & Gossip Protocols (Slides 27–34).
- Module 4: Consistency Protocols, Primary-Backup & Quorum Systems (Slides 35–38).
- Module 5: Real-World Case Studies, Review & Practical Lab Specifications (Slides 39–40).

Slide 35: Primary-Based Protocols: Remote-Write & Local-Write Primary-Backup

Remote-Write Primary-Backup

- **Single Primary Coordinator**: All write operations are routed to a designated Primary Server P.
- **Synchronous Propagation**: Primary P executes update locally, multicasts update to Backups B1..Bk, and waits for ACKs.
- **Consistency**: Guarantees Sequential Consistency; writes incur wide-area round-trip latency.
- **Use Case**: Enterprise SQL databases (PostgreSQL streaming replication).

Local-Write Primary-Backup Migration

- **Dynamic Role Transfer**: Primary server status migrates dynamically to whichever node needs to perform writes.
- **Local Write Execution**: Server S1 fetches primary state from S0, becomes New Primary, and executes writes locally.
- **Use Case**: Mobile workers who require local primary write capabilities while disconnected.

Slide 36: Replicated-Write Protocols: Active Replication & SMR

Active Replication Concept

- **Direct Multi-Master Broadcast**: Writes are transmitted directly to ALL replicas simultaneously, bypassing single primaries.
- **Equal Processing**: Every replica executes incoming update operations independently.

State Machine Replication (SMR) Principles

- **Deterministic State Machines**: Replicas host identical deterministic state machines.
- **Totally-Ordered Multicast**: Operations must be delivered to all replicas in the EXACT same order using consensus engines (Paxos/Raft).
- **Determinism Invariant**: Given identical initial state and identical ordered operations, all replicas produce identical output.

Slide 37: Quorum-Based Protocols: Gifford's Majority Voting & Mathematics

Gifford's Voting Concept

• **Quorum Voting**: Clients must secure permissions from a quorum (subset) of replicas before executing reads or writes.

• **Cluster Parameters**: N total replicas (1 vote each). N_R = Read Quorum votes required. N_W = Write Quorum votes required.

Gifford's Two Overlap Rules

• **Rule 1 (Read-Write Overlap)**: $N_R + N_W > N \rightarrow$ Guarantees read quorum and write quorum overlap by ≥ 1 replica holding latest version.

• **Rule 2 (Write-Write Overlap)**: $N_W > \frac{N}{2} \rightarrow$ Guarantees two write quorums overlap by ≥ 1 replica, preventing split-brain writes.

• **Fault Tolerance**: System tolerates up to $N - N_W$ write failures and $N - N_R$ read failures.

Slide 38: Analyzing Quorum System Configurations (N = 5 Replicas)

Configuration Type	Quorum Parameters	Rule Checks & Performance Evaluation
Read-Heavy (Fast Reads)	NR = 1, NW = 5	NR + NW = 6 > 5 (Valid); NW = 5 > 2.5 (Valid). Reads require 1 replica (fast!); Writes require all 5 (slow).
Balanced Majority Quorum	NR = 3, NW = 3	NR + NW = 6 > 5 (Valid); NW = 3 > 2.5 (Valid). Balanced performance; tolerates 2 server crashes.
INVALID Configuration	NR = 2, NW = 2	NR + NW = 4 <= 5 (INVALID!). Fails Rule 1! Reads do not overlap writes and return stale data.

Content

- Module 1: Introduction & Data-Centric Consistency Models (Slides 3–16).
- Module 2: Client-Centric Consistency Models & Session Guarantees (Slides 17–26).
- Module 3: Replica Management, Content Placement & Gossip Protocols (Slides 27–34).
- Module 4: Consistency Protocols, Primary-Backup & Quorum Systems (Slides 35–38).
- Module 5: Real-World Case Studies, Review & Practical Lab Specifications (Slides 39–40).

Slide 39: Cache-Coherence Protocols & Caching Execution Modes

Cache-Coherence Protocols

- Cache-Invalidate Protocols: Master broadcasts short invalidation signals on write. Clients invalidate local cache entries. Efficient for high write volume.
- Cache-Update Protocols: Master broadcasts updated data payload to active client caches. Eliminates cache misses for subsequent reads.

Caching Execution Modes

- Write-Through Caching: Client updates local cache, which synchronously forwards update to master store. High consistency; write blocks.
- Write-Back (Write-Behind) Caching: Client updates local cache and returns instantly. Cache flushes updates asynchronously in background batches. High speed; risk of data loss on crash.

Slide 40: Real-World Case Studies: Apache Cassandra & Google Bigtable

Apache Cassandra Tunable Consistency

- Decentralized P2P Architecture: Ring topology using Dynamo-style partitioning and gossip anti-entropy.
- Tunable Consistency Levels: Developers specify Read/Write consistency per query (ONE, QUORUM, ALL, LOCAL_QUORUM).
- Quorum Flexibility: Setting Read=QUORUM and Write=QUORUM guarantees strong consistency across cluster nodes.

Google Bigtable & Chubby Lock Service

- Bigtable Architecture: Distributed structured storage guaranteeing single-row strict consistency via log commits.
- Chubby Lock Service: Provides coarse-grained distributed locking and master election using Paxos consensus across 5 replicas.

Recommended Supplementary Resources

1. Data-Centric Consistency Models & Foundations

- **Lamport, L. (1979)**. *"How to Make a Multiprocessor Computer That Correctly Executes Multiprocess Programs"*. *IEEE Transactions on Computers*, C-28(9), 690–691.
  - The classic paper that formally defined **Sequential Consistency**, establishing the interleaved program-order model used across distributed systems and shared-memory architectures.
- **Ahamad, M., Neiger, G., Burns, J. E., & Hutto, P. W. (1995)**. *"Causal Memory: Definitions, Implementation, and Programming"*. *Distributed Computing*, 9(1), 37–49.
  - Formally defines **Causal Consistency** and compares its performance and semantic guarantees against sequential consistency.
- **Gharachorloo, K., Lenoski, D., Koufaty, J., et al. (1990)**. *"Memory Consistency and Event Order in Scalable Shared-Memory Multiprocessors"*. *Proceedings of ISCA '90*, 15–26.
  - Introduces **Release Consistency** (Acquire/Release primitives) and Lazy Release Consistency protocols.

2. Client-Centric & Eventual Consistency

- **Terry, D. B., Theimer, M. M., Petersen, K., Welch, A. J., & Hauser, C. D. (1994)**. *"Session Guarantees for Weakly Consistent Replicated Data"*. *Proceedings of PDIS '94*, 140–149.
  - The foundational paper that defined the four **Client-Centric Consistency Guarantees** (Monotonic Reads, Monotonic Writes, Read-Your-Writes, Writes-Follow-Reads) for mobile and disconnected computing.
- **Vogels, W. (2009)**. *"Eventually Consistent"*. *Communications of the ACM*, 52(1), 40–44.
  - Amazon's CTO explains the practical architectural trade-offs of **Eventual Consistency** in high-scale cloud platforms and web services.
- **Abadi, D. J. (2012)**. *"Consistency Tradeoffs in Modern Distributed Database System Design: CAP is Only Part of the Story"*. *Computer*, 45(2), 37–42.
  - Introduces the **PACELC Theorem**, extending CAP to account for Latency vs. Consistency trade-offs during normal execution.

Recommended Supplementary Resources

3. Quorum Systems & Replicated-Write Protocols

- **Gifford, D. K. (1979)**. *"Weighted Voting for Replicated Data"*. *Proceedings of SOSP '79*, 150–162.
  - The seminal paper introducing **Quorum-Based Voting** and the overlap rules (\(N_R + N_W > N\) and \(N_W > N/2\)) for tuning read/write performance.
- **Schneider, F. B. (1990)**. *"Implementing Fault-Tolerant Services Using the State Machine Approach: A Tutorial"*. *ACM Computing Surveys*, 22(4), 299–319.
  - The definitive tutorial on **Active Replication** and **State Machine Replication (SMR)** using totally-ordered multicast.

4. Gossip, Epidemic Protocols & Real-World Systems

- **Demers, A., Greene, D., Hauser, C., Irish, W., Larson, J., et al. (1987)**. *"Epidemic Algorithms for Replicated Database Maintenance"*. *Proceedings of PODC '87*, 1–12.
  - The classic paper introducing **Anti-Entropy** (Push, Pull, Push-Pull) and **Rumor Spreading** (gossip) protocols, along with **Death Certificates (Tombstones)** for distributed deletions.
- **DeCandia, G., Hastorun, D., Jampani, M., et al. (2007)**. *"Dynamo: Amazon's Highly Available Key-Value Store"*. *Proceedings of SOSP '07*, 205–220.
  - Examines how Amazon combined vector clocks, gossip protocols, hint handoff, tombstones, and quorum voting (\(N, R, W\)) to build a highly available eventually consistent store.
- **Lakshman, A., & Malik, P. (2010)**. *"Cassandra: A Decentralized Structured Storage System"*. *ACM SIGOPS Operating Systems Review*, 44(2), 35–40.
  - Details Apache Cassandra's architecture and its implementation of **Tunable Consistency** (`ONE`, `QUORUM`, `ALL`, `LOCAL_QUORUM`).